BÁO CÁO THỰC HÀNH LAB 7

Phân tích Động: Phủ mã (Code Coverage) & Giám sát Thực thi

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 12/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 7 nghiên cứu về phương pháp phân tích động (Dynamic Analysis) thông qua kỹ thuật đo lường độ phủ mã nguồn (Code Coverage) và theo dõi thực thi chương trình trên module Order Processing (Xử lý đơn hàng) của hệ thống SecureSys:

1. **Đo lường độ phủ mã nguồn bằng công cụ tiêu chuẩn công nghiệp gcov:** Nắm vững cơ chế biên dịch chèn mã giám sát (Instrumentation) bằng cờ `--coverage` của GCC để tự động thu thập dữ liệu thực thi (tệp `.gcno` và `.gcda`), từ đó tính toán chính xác tỷ lệ độ phủ dòng lệnh (Statement Coverage) và phát hiện các dòng lệnh chưa từng được thực thi.
2. **Hiểu sâu sắc sự khác biệt giữa "Tỷ lệ test PASS 100%" và "Coverage cao":** Chứng minh trực quan bằng thực nghiệm rằng một bộ kiểm thử có thể đạt tỷ lệ 100% test case thành công nhưng vẫn có độ phủ rất thấp (bỏ sót hàng loạt nhánh rẽ logic và lỗ hổng tiềm ẩn).
3. **Áp dụng kỹ thuật phân hoạch lớp tương đương (Equivalence Partitioning) và phân tích giá trị biên (Boundary Value Analysis):** Chuyển dịch từ việc viết test case ngẫu nhiên sang việc thiết kế bộ kiểm thử có ý thức cấu trúc, tối ưu hóa không gian tham số đầu vào.
4. **Sinh test case tổ hợp tự động (Combinatorial Testing):** Tự động hóa quá trình sinh các ca kiểm thử bao phủ toàn bộ tích Descartes của các lớp tương đương, đảm bảo mọi nhánh rẽ lồng nhau đều có ít nhất một ca kiểm thử đại diện.
5. **Xây dựng Test Oracle độc lập:** Cài đặt hàm dự đoán kết quả mong đợi (Oracle) viết bằng Python hoàn toàn tách biệt khỏi mã C, tuân thủ nguyên tắc kiểm thử chuẩn mực nhằm tránh cạm bẫy "kiểm thử chương trình bằng chính nó".

2. ĐẶC TẢ MODULE ORDER PROCESSING VÀ CÂY QUYẾT ĐỊNH (DECISION TREE)

2.1. Đặc tả logic nghiệp vụ của hàm `process_order()`

Hàm `process_order(int quantity, double unit_price, int is_member, OrderResult *out)` thực hiện tính toán chiết khấu và kiểm tra các điều kiện an toàn giao dịch:

- **Kiểm tra số lượng hợp lệ:** Số lượng `quantity` phải thỏa mãn `1 ≤ quantity ≤ 100`. Nếu `quantity ≤ 0` hoặc `quantity > 100`, hàm trả về ngay mã lỗi `ORDER_INVALID_QTY`.
- **Kiểm tra đơn giá hợp lệ:** Đơn giá `unit_price` phải là số dương (`unit_price > 0.0`). Nếu `unit_price ≤ 0.0`, hàm trả về mã lỗi `ORDER_INVALID_PRICE`.
- **Chính sách chiết khấu (Discount Policy):**

- Khách hàng thành viên (`is_member == 1`): Nếu mua sỉ từ 10 sản phẩm trở lên (`quantity ≥ 10`) → giảm giá 15% (`discount = 0.15`); nếu mua lẻ dưới 10 sản phẩm → giảm giá 5% (`discount = 0.05`).
- Khách hàng vãng lai (`is_member == 0`): Nếu mua sỉ từ 20 sản phẩm trở lên (`quantity ≥ 20`) → giảm giá 10% (`discount = 0.10`); nếu mua lẻ dưới 20 sản phẩm → không giảm giá (`discount = 0.00`).

- **Ngưỡng phê duyệt giao dịch (Fraud Control / Approval Threshold):**
  Tổng tiền sau giảm giá được tính theo công thức: `total = quantity × unit_price × (1.0 - discount)`.
  Nếu `total > 10,000,000` VND (ngưỡng 10 triệu đồng), giao dịch bị đánh dấu là đơn hàng bất thường cần xét duyệt và trả về mã `ORDER_APPROVAL_REQUIRED`. Ngược lại, trả về `ORDER_OK`.

2.2. Cây quyết định (Decision Tree) và các đường thực thi (Execution Paths)

Cấu trúc rẽ nhánh lồng nhau của `process_order()` tạo thành một cây quyết định với 8 đường thực thi nghiệp vụ cốt lõi:

```
[GỐC] Nhập vào (quantity, unit_price, is_member)
  |
  +-- (quantity <= 0 || quantity > 100) ?
  |     +-- [ĐÚNG] ----------------------------------> [Đường 1] return ORDER_INVALID_QTY
  |
  +-- (unit_price <= 0.0) ?
  |     +-- [ĐÚNG] ----------------------------------> [Đường 2] return ORDER_INVALID_PRICE
  |
  +-- is_member == 1 ?
  |     |-- quantity >= 10 ?
  |     |     +-- [ĐÚNG] ----------------------------> [Đường 3] discount = 0.15 (VIP mua sỉ)
  |     |     +-- [SAI]  ----------------------------> [Đường 4] discount = 0.05 (VIP mua lẻ)
  |
  +-- is_member == 0 ?
  |     |-- quantity >= 20 ?
  |     |     +-- [ĐÚNG] ----------------------------> [Đường 5] discount = 0.10 (Khách sỉ)
  |     |     +-- [SAI]  ----------------------------> [Đường 6] discount = 0.00 (Khách lẻ)
  |
  \-- (total > 10,000,000) ?
        |-- [ĐÚNG] ----------------------------------> [Đường 7] return ORDER_APPROVAL_REQUIRE
D
        \-- [SAI]  ----------------------------------> [Đường 8] return ORDER_OK
```

Nhận xét: Để bao phủ toàn bộ các hành vi nghiệp vụ, một bộ kiểm thử bắt buộc phải kích hoạt đầy đủ cả 8 đường thực thi này.

3. SO SÁNH ĐỊNH LƯỢNG BA BỘ TEST (PARTIAL VS. FULL VS. COMBINATORIAL)

Thực nghiệm được tiến hành trên 3 bộ kiểm thử khác nhau bằng công cụ `python/coverage_runner.py` kết hợp `gcov`:

Bộ kiểm thử (Test Suite)	Số lượng Test Case	Tỷ lệ PASS	Số dòng thực thi (gcov)	Độ phủ mã (Coverage)	Đánh giá
Partial Suite *(Không đầy đủ)*	2 ca	**100% (2/2)**	51 / 64 dòng	**79.69%**	Chưa đạt
Full Suite *(Đầy đủ ca biên)*	9 ca	**100% (9/9)**	59 / 64 dòng	**92.19%**	Tốt (+12.5%)
Combinatorial Suite *(Tổ hợp tự sinh)*	**56 ca**	**100% (56/56)**	59 / 64 dòng	**92.19%**	Tối ưu

Phát hiện mấu chốt về bản chất kiểm thử:
- Bộ test `partial` chỉ có 2 ca kiểm thử và **cả 2 ca đều PASS 100%**. Nếu chỉ nhìn vào báo cáo "All tests passed", lập trình viên sẽ có ảo tưởng nguy hiểm rằng mã nguồn đã an toàn và hoàn hảo.
- Tuy nhiên, độ phủ của bộ `partial` chỉ đạt 79.69%, bỏ sót hoàn toàn: 1. Nhánh bắt lỗi số lượng không hợp lệ (`quantity ≤ 0` và `quantity > 100`). 2. Nhánh bắt lỗi đơn giá âm hoặc bằng 0 (`unit_price ≤ 0`). 3. Nhánh chiết khấu cho khách hàng không phải thành viên (`is_member == 0`). 4. Nhánh cảnh báo đơn hàng lớn vượt ngưỡng phê duyệt (`total > 10,000,000`).
- Khi bổ sung 7 ca kiểm thử biên trong bộ `full`, độ phủ tăng vọt thêm **12.5 điểm phần trăm** (từ 79.69% lên 92.19%), kích hoạt toàn bộ các nhánh nghiệp vụ còn thiếu.

4. PHÂN TÍCH CHI TIẾT CÁC DÒNG KHÔNG THỂ PHỦ (7.81% CÒN LẠI)

Dù đã tăng số ca kiểm thử lên 56 ca trong bộ tổ hợp, độ phủ mã vẫn dừng lại ở mức **92.19%** (59/64 dòng). Dưới đây là phân tích chi tiết 5 dòng mã lệnh không thể thực thi:

Số dòng	Mã nguồn tương ứng	Nguyên nhân kỹ thuật không thể phủ
Dòng 92	`default: return "UNKNOWN";`	Trong hàm `order_status_to_string()`, biến kiểu `OrderStatus` là enum chỉ nhận các giá trị hợp lệ từ 0 đến 3 (`OK`, `INVALID_QTY`, `INVALID_PRICE`, `APPROVAL_REQUIRED`). Nhánh `default` là kỹ thuật *lập trình phòng thủ (defensive programming)* để tránh cảnh báo biên dịch của GCC, không bao giờ được chạm tới trong luồng chạy bình thường.

Dòng 101, 102	`if (argc < 2) {`
	`  fprintf(stderr, "Cach dung...");`
	`  return 1;`
	`}`
		Đây là nhánh kiểm tra số lượng đối số dòng lệnh của hàm `main()`. Khi chạy kiểm thử tự động, chương trình CLI luôn được gọi kèm đường dẫn file CSV (luôn có `argc == 2`). Muốn phủ dòng này, cần một test case riêng biệt ở tầng CLI thực thi lệnh không có tham số.

Dòng 106, 107	`if (!f) {`
	`  fprintf(stderr, "[LOI] Khong mo duoc...");`
	`  return 1;`
	`}`
		Nhánh xử lý ngoại lệ khi hàm `fopen()` thất bại. Do tệp dữ liệu test CSV luôn tồn tại hợp lệ trong thư mục `dataset/`, lỗi không mở được tệp không bao giờ xảy ra trong quá trình chạy bộ test nghiệp vụ.

Kết luận: Cả 5 dòng chưa được phủ đều nằm ngoài phạm vi logic nghiệp vụ của hàm `process_order()`. Bản thân hàm `process_order()` đã đạt **độ phủ tuyệt đối 100%** trên toàn bộ các câu lệnh và nhánh rẽ.

5. KỸ THUẬT SINH TEST TỔ HỢP TỰ ĐỘNG VÀ VAI TRÒ CỦA ORACLE ĐỘC LẬP

5.1. Khái niệm Test Oracle và Cạm bẫy "Kiểm thử chương trình bằng chính nó"

Một **Test Oracle** là cơ chế xác định xem kết quả thực thi của một ca kiểm thử có đúng với yêu cầu hay không. Sai lầm kinh điển và nguy hiểm nhất trong kiểm thử phần mềm là sử dụng chính chương trình đang kiểm thử để sinh kết quả mong đợi (ví dụ: chạy chương trình C một lần, ghi lại kết quả ra file, rồi lấy file đó làm chuẩn so sánh cho các lần chạy sau).

Nếu trong mã C có sẵn một lỗi logic (ví dụ: tính sai thuế hoặc sai mức giảm giá), chương trình sẽ liên tục "tự khen mình đúng" và toàn bộ các bộ test đều báo PASS giả tạo!

5.2. Cài đặt Oracle độc lập trong Lab 7

Trong `python/test_case_generator.py`, nhóm thực hành xây dựng hàm `expected_status()` hoàn toàn độc lập:

```
def expected_status(qty: int, price: float, is_member: int) -> str:
    if qty <= 0 or qty > 100: return "INVALID_QTY"
    if price <= 0:            return "INVALID_PRICE"
    if is_member:
        discount = 0.15 if qty >= 10 else 0.05
    else:
        discount = 0.10 if qty >= 20 else 0.0
    total = qty * price * (1 - discount)
    if total > 10_000_000:    return "APPROVAL_REQUIRED"
    return "OK"
```

Tính ưu việt của Oracle độc lập:

- Được viết bằng ngôn ngữ bậc cao (Python) thay vì C, loại trừ hoàn toàn các lỗi quản lý bộ nhớ hoặc ép kiểu dữ liệu.
- Được hiện thực trực tiếp từ tài liệu đặc tả yêu cầu (SRS), không sao chép cấu trúc mã nguồn của C.
- Tự động dán nhãn kết quả mong đợi cho hàng nghìn ca kiểm thử mới sinh ra mà không cần con người can thiệp thủ công.

6. BỔ SUNG LỚP TƯƠNG ĐƯƠNG BIÊN MỚI (BƯỚC 5)

Thực hiện yêu cầu Bước 5 của đề bài, nhóm thực hành đã bổ sung thêm một lớp tương đương biên mới: `("valid_just_below_member", 9)` đại diện cho giá trị ngay sát dưới ngưỡng được hưởng ưu đãi sỉ của thành viên (ngưỡng 10 sản phẩm):

```
--- a/python/test_case_generator.py
+++ b/python/test_case_generator.py
@@ -19,6 +19,7 @@ QUANTITY_CLASSES = [
     ("invalid_low", 0),
     ("invalid_high", 101),
     ("valid_low", 5),
+    ("valid_just_below_member", 9),
     ("valid_member_threshold", 10),
     ("valid_nonmember_threshold", 20),
     ("valid_max", 100),
```

Kết quả sau khi mở rộng:

- Số lớp tương đương của `quantity` tăng từ 6 lên **7 lớp**.
- Tổng số ca kiểm thử sinh ra theo tích Descartes tăng từ 48 lên **56 ca kiểm thử** ($7 imes 4 imes 2 = 56$).
- Khi chạy bộ kiểm thử 56 ca với `coverage_runner.py`, **100% (56/56) ca kiểm thử đều PASS** và độ phủ mã duy trì ở mức tối đa **92.19%**.
- Ca kiểm thử `qty = 9` chứng minh rằng hệ thống áp dụng chính xác mức giảm giá 5% cho thành viên và không bị nhầm lẫn bởi lỗi lệch một (Off-by-one).

7. BẢNG SO SÁNH: KIỂM THỬ TĨNH (LAB 4, 5, 6) VS. KIỂM THỬ ĐỘNG (LAB 3, 7)

Tiêu chí so sánh	Kiểm thử Tĩnh (Static Analysis / SMT / BMC)	Kiểm thử Động (Dynamic Testing / Coverage / ASan)
Bản chất thực thi	Phân tích mô hình toán học hoặc mã nguồn mà không chạy chương trình.	Bắt buộc biên dịch thành mã máy và thực thi với dữ liệu đầu vào cụ thể.
Độ bao phủ (Coverage)	Xem xét mọi đường đi tiềm năng trên cây cú pháp AST hoặc đồ thị CFG.	Chỉ bao phủ các dòng lệnh và nhánh rẽ được kích hoạt bởi bộ test case.
Báo động giả (False Positives)	Có thể xuất hiện do sự trừu tượng hóa quá mức hoặc điểm mù con trỏ.	**Tuyệt đối không có báo động giả**; vi phạm xảy ra trực tiếp trên CPU/RAM.
Chi phí tính toán	Có thể bùng nổ tổ hợp khi đồ thị trạng thái hoặc công thức logic quá lớn.	Tuyến tính theo số lượng test case; dễ dàng mở rộng và phân tán.
Vai trò bổ trợ	Phát hiện sớm lỗi thiết kế, backdoor tiềm ẩn và chứng minh tính tương đương.	Đo lường độ tin cậy thực tế, phát hiện lỗi tràn bộ nhớ và thẩm định nghiệp vụ.

8. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_coverage.py` kiểm tra 4 tiêu chí cốt lõi:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	`test_partial_suite_all_pass`	Xác nhận 2/2 test của bộ partial đều PASS	PASS
2	`test_full_suite_all_pass`	Xác nhận 9/9 test của bộ full đều PASS	PASS
3	`test_full_covers_significantly_more`	Xác nhận bộ full tăng độ phủ ≥ 5% so với partial (+12.5%)	PASS
4	`test_combinatorial_suite_matches_or_exceeds`	Xác nhận 56/56 test tổ hợp PASS và đạt độ phủ ≥ full	PASS

Thực thi lệnh kiểm thử: `python tests/test_coverage.py` → Kết quả: **4/4 test đã PASS (100%)**.

9. TỔNG KẾT BÀI THỰC HÀNH VÀ BÀI HỌC KINH NGHIỆM

1. Làm chủ kỹ thuật phân tích động và đo độ phủ mã nguồn bằng `gcov`, nắm vững cách đọc hiểu báo cáo độ phủ dòng lệnh và độ phủ nhánh.
2. Thấu hiểu sâu sắc rằng tỷ lệ test PASS 100% không bảo đảm an toàn phần mềm nếu thiếu độ phủ mã tương ứng; chất lượng của một bộ kiểm thử được đo lường bằng khả năng bao quát các nhánh rẽ và trường hợp biên.
3. Nắm vững phương pháp thiết kế bộ kiểm thử chuyên nghiệp dựa trên Phân hoạch lớp tương đương (Equivalence Partitioning) và Phân tích giá trị biên (Boundary Value Analysis).
4. Làm chủ kỹ thuật xây dựng Test Oracle độc lập để tự động hóa hoàn toàn quy trình sinh và thẩm định hàng nghìn ca kiểm thử trong đường ống CI/CD hiện đại.